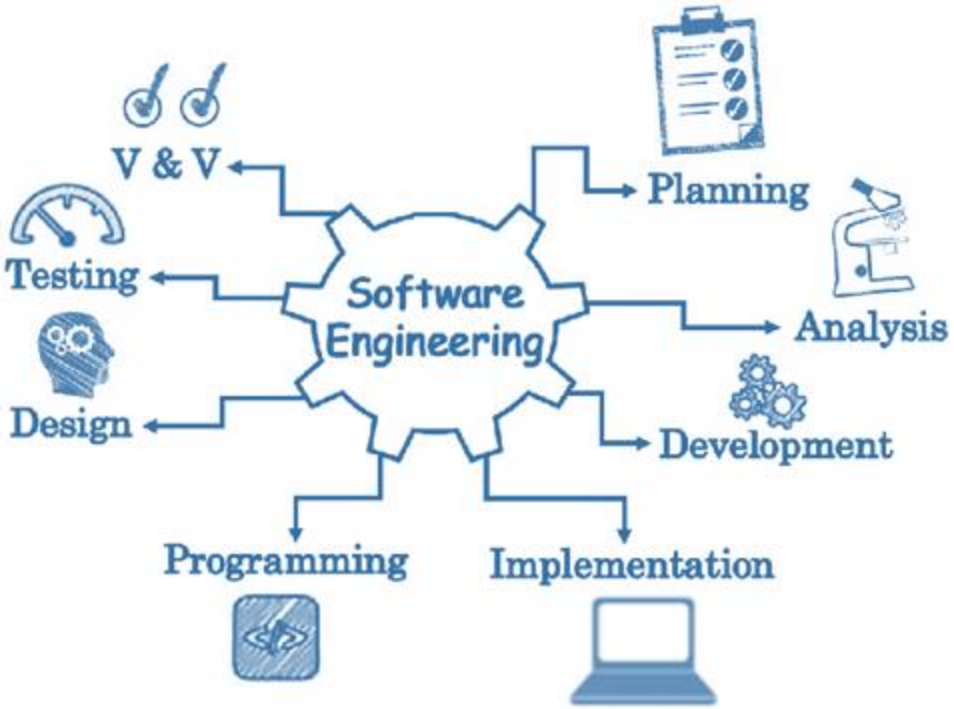
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Software Process Models and Project

5.1 Reuse Concepts

- Solution Objects and Reusable Components
- Specification vs. Implementation Inheritance
- Delegation and the Liskov Substitution Principle

5.2 Common Design Patterns

- Creational Patterns: Factory, Abstract Factory
- Structural Patterns: Adapter, Composite, Bridge
- Behavioral Patterns: Strategy, Command

5.3 Selecting and Applying Design Patterns

- Heuristics for Pattern Selection
- Integrating Patterns into Designs

5.4 Managing Reuse

- Documenting Reuse Strategies
- Assigning Responsibilities for Reusable Components

Application domain vs solution domain objects

- ***Application objects***, also called ***domain objects***, represent concepts of the domain that are relevant to the system.
  - They are identified by the application domain specialists and by the end users.
- ***Solution objects*** represent concepts that do not have a counterpart in the application domain,
  - They are identified by the developers
  - Examples: Persistent data stores, user interface objects, middleware.

Application domain vs solution domain objects

- *Modeling with UML*, class diagrams can be used to model both the application domain and the solution domain.
  - **Application objects**, also called "domain objects," represent concepts of the domain that are relevant to the system.
  - **Solution objects** represent components that do not have a counterpart in the application domain, such as persistent data stores, user interface objects, or middleware.
- During analysis, we identify **entity objects** and their relationships, attributes, and operations.
- Most entity objects are **application objects** that are independent of any specific system.
- During analysis, we also identify **solution objects** that are visible to the user, such as boundary and control objects representing forms and transactions defined by the system.
- During **system design**, we identify more solution objects in terms of software and hardware platforms.
- During **object design**, we refine and detail both application and solution objects and identify additional solution objects needed to bridge the object design gap.



Object Design

- Object design is the process of adding details to the requirements analysis and making implementation decisions
- The object designer must choose among different ways to implement the analysis model with the goal to minimize execution time, memory and other measures of cost.

- **Requirements Analysis**: Use cases, functional and dynamic model deliver operations for object model
- **Object Design**:

  - Iterates on the models, in particular the object model and refine the models;

  - Serves as the basis of implementation

Example of Object Design Activities

- Identification of existing components
- Full definition of associations
- Full definition of classes:
  - System Design => Service,
  - Object Design => API
- Specifying the contract for each component
- Choosing algorithms and data structures

- Identifying possibilities of reuse
- Detection of solution-domain classes
- Optimization
- Increase of inheritance
- Decision on control
- (Re-)Packaging

Example of Object Design Activities

1. Reuse: Identification of existing solutions
   - Use of inheritance
   - Off-the-shelf components and additional solution objects
   - Design patterns
2. Interface specification
   - Describes precisely each class interface

Object Design

3. Object model restructuring
   - Transforms the object design model to improve its understandability and extensibility
4. Object model optimization
   - Transforms the object design model to address *performance* criteria such as response time or memory utilization.

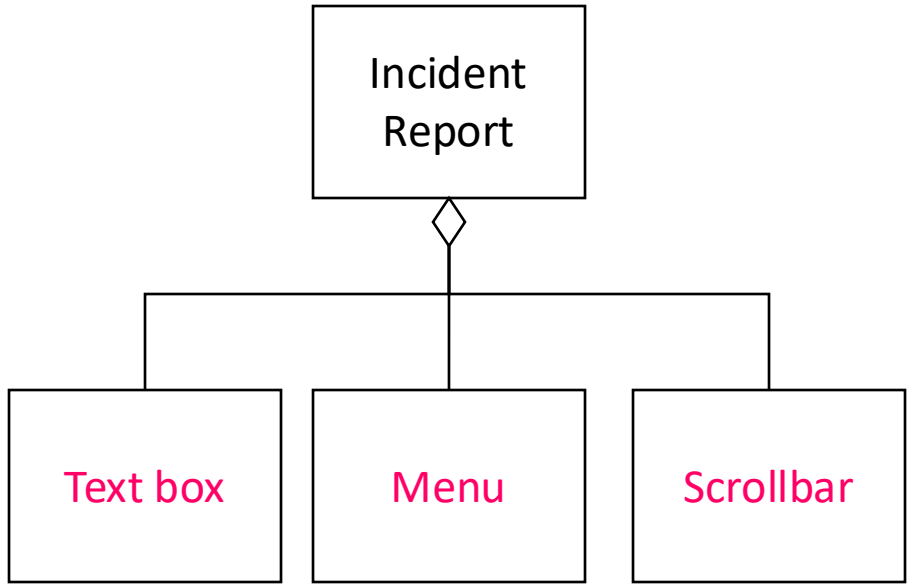
Mapping Models to Code

Example of Object Design Activities

Requirements Analysis
(Language of Application Domain)

Incident Report

Object Design
(Language of Solution Domain)

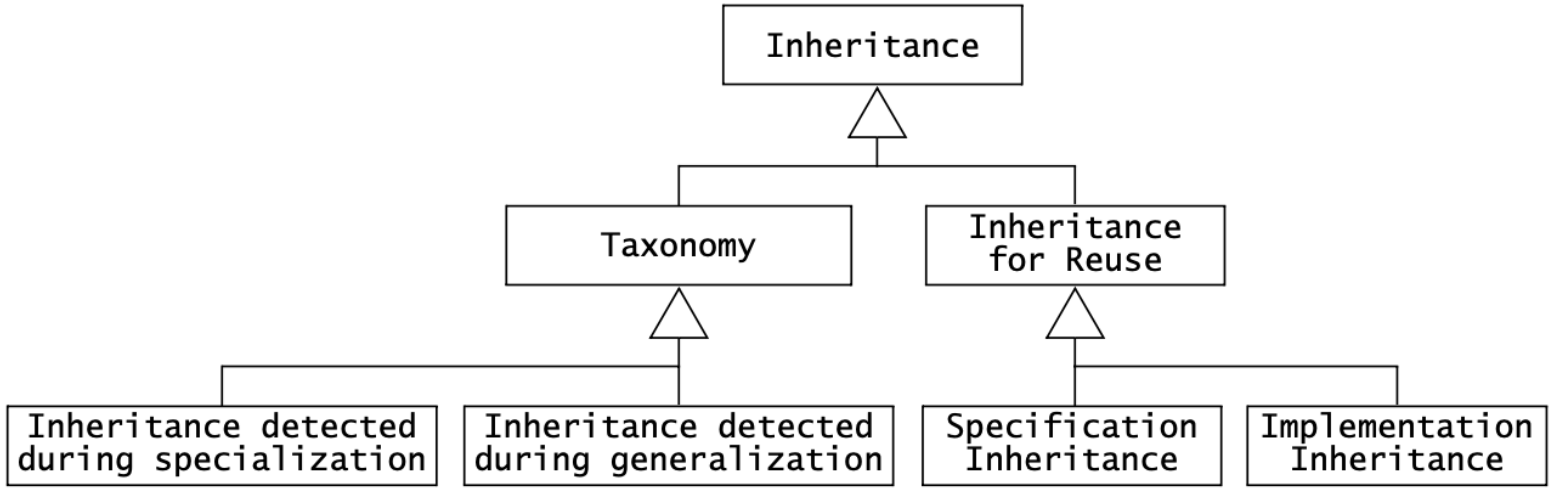


Do the two Incident Reports have the same meaning?

The Use of Inheritance

- Inheritance is used to achieve two different goals
  - Description of Taxonomies
  - Interface Specification
- Identification of taxonomies
  - Used during requirements analysis.
  - Activity: identify application domain objects that are hierarchically related
  - Goal: make the analysis model more *understandable*
- Service specification
  - Used during object design
  - Activity:
  - Goal: increase *reusability*, enhance modifiability and extensibility
- Inheritance is found either by specialization or generalization

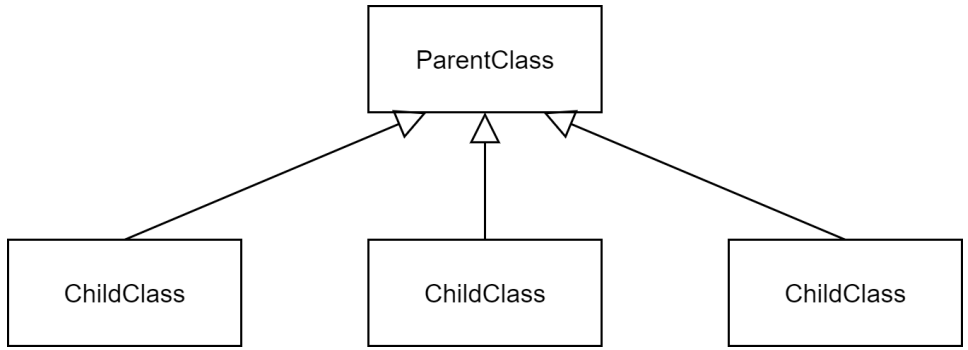
The Use of Inheritance



- When modeling **taxonomies**, the inheritance relationships can be identified either during specializations (when specialized classes are identified after general ones) or during generalizations (when general classes are abstracted out of a number of specialized ones).
- When using inheritance **for reuse**, specification inheritance represents subtyping relationships, and implementation inheritance represents reuse among conceptually unrelated classes.

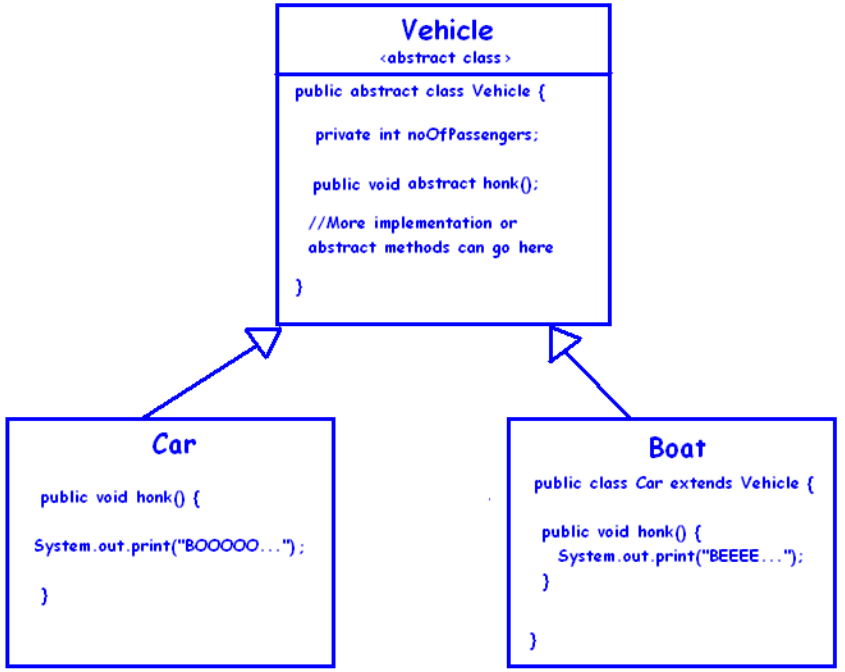
Specification Inheritance and Implementation Inheritance

- The focus of **generalization** (i.e., identifying a common superclass from a number of existing classes) and **specialization** (i.e., identifying new subclasses given an existing superclass) is to organize analysis objects into an understandable hierarchy.
- Readers of the analysis model can start from the abstract concepts, grasp the core functionality of the system, and make their way down to concrete concepts and review specialized behavior.



Specification Inheritance and Implementation Inheritance

- The focus of inheritance during **object design** is to reduce **redundancy** and enhance **extensibility**.
- By factoring all redundant behavior into a single superclass, we reduce the risk of introducing **inconsistencies** during changes (e.g., when repairing a defect) since we have to make changes only once for all subclasses.
- By providing **abstract classes and interfaces** that are used by the application, we can write new specialized behavior by writing new subclasses that comply with the abstract interfaces.
- For example, we can write an application for the different types of vehicle in terms of an *abstract vehicle*:
  - which defines all the operations that all vehicle should support, and a series of specialized classes for each type of vehicle (e.g., car, truck, boat)
  - When we need to extend the application to a new vehicle type, we only need to add a new specialized class.



Specification Inheritance and Implementation Inheritance

- Implementation inheritance
  - Also called class inheritance
  - Goal: Extend an applications' functionality by reusing functionality in parent class
  - Inherit from an existing class with some or all operations already implemented

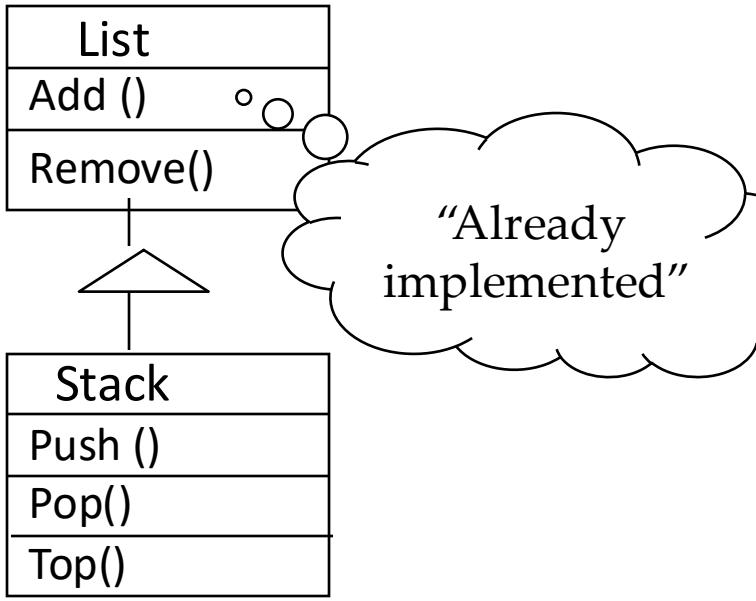
- ***Interface*** inheritance
  - Also called subtyping
  - Inherit from an abstract class with all operations specified, but not yet implemented

Liskov Substitution Principle

- **Liskov Substitution Principle**
  - If an object of type S can be substituted in all the places where an object of type T is expected, then S is a subtype of T.
- **Interpretation**
  - In object design, the ***Liskov Substitution Principle*** means that if all classes are subtypes of their superclasses, all inheritance relationships are ***specification*** inheritance relationships.
  - In other words, a method written in terms of a superclass T must be able to use instances of any subclass of T without knowing whether the instances are of a subclass.
  - Consequently, new subclasses of T can be added without modifying the methods of T, hence leading to an extensible system.
  - An inheritance relationship that complies with the Liskov Substitution Principle is called **strict inheritance**.

Specification Inheritance and Implementation Inheritance

- A very similar class is already implemented that does almost the same as the desired class implementation.
  - Example: I have a List class, I need a Stack class. How about subclassing the Stack class from the List class and providing three methods, Push() and Pop(), Top()?



Why Avoid Subclassing List?

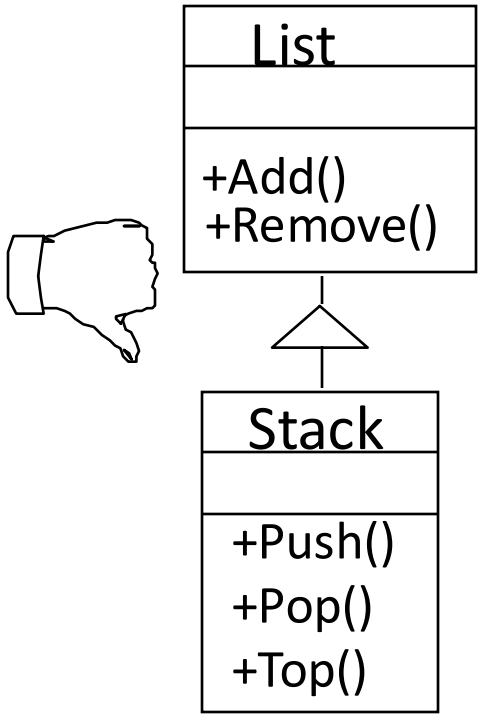
•**Violates Liskov Substitution Principle (LSP):** A Stack is not a subtype of List, as it shouldn't expose methods like Insert() or RemoveAt(), which could break the stack's behavior.

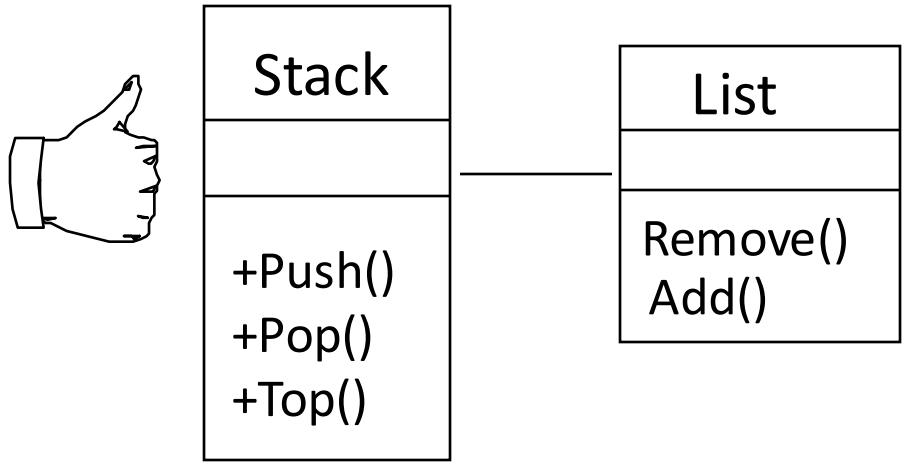
•**Unsafe Exposure:** Inheriting from List would expose all List methods (e.g., arbitrary insertions/deletions), allowing users to bypass stack operations and corrupt the data structure.

An elephant becomes a car!

Delegation

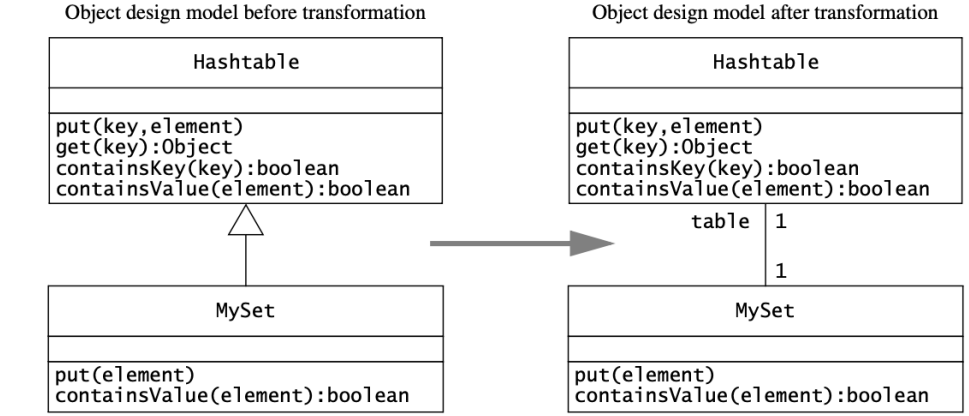
- **Inheritance**: Extending a Base class by a new operation or overwriting an operation.
- **Delegation**: Catching an operation and sending it to another object.
- Which of the following models is better for implementing a stack?





Delegation

- **Delegation** is the alternative to **implementation** inheritance that should be used when reuse is desired.
- A class is said to delegate to another class if it implements an operation by resending a message to another class.
- Delegation makes explicit the dependencies between the reused class and the new class.




```
/* Implementation of MySet using
inheritance */
class MySet extends Hashtable {
    /* Constructor omitted */
    MySet() {
    }

    void put(Object element) {
        if (!containsKey(element)){
            put(element, this);
        }
    }
    boolean containsValue(Object
            element){
        return containsKey(element);
    }
    /* Other methods omitted */
}
```

```
/* Implementation of MySet using
delegation */
class MySet {
    private Hashtable table;
    MySet() {
        table = Hashtable();
    }
    void put(Object element) {
        if (!containsValue(element)){
            table.put(element,this);
        }
    }
    boolean containsValue(Object
            element) {
        return
        (table.containsKey(element));
    }
    /* Other methods omitted */
}
```

Figure 8-3 An example of implementation inheritance. The left column depicts a questionable implementation of `MySet` using implementation inheritance. The right column depicts an improved implementation using delegation (UML class diagram and Java).

Delegation vs Implementation Inheritance

- Delegation
  - Pro:
    - Flexibility: Any object can be replaced at run time by another one (as long as it has the same type)
  - Con:
    - Inefficiency: Objects are encapsulated.
- Inheritance
  - Pro:
    - Straightforward to use
    - Supported by many programming languages
    - Easy to implement new functionality
  - Con:
    - Inheritance exposes a subclass to the details of its parent class
    - Any change in the parent class implementation forces the subclass to change (which requires recompilation of both)

Design Pattern

A Game: Get-15

- Start with the nine numbers 1,2,3,4, 5, 6, 7, 8 and 9.
- You and your opponent take alternate turns, each taking a number
- Each number can be taken only once: If you opponent has selected a number, you cannot also take it.
- The first person to have any three numbers that total 15 wins the game.
- Example:

You: 1 5 3 8

Opponent: 6 9 7 2

Opponent Wins!

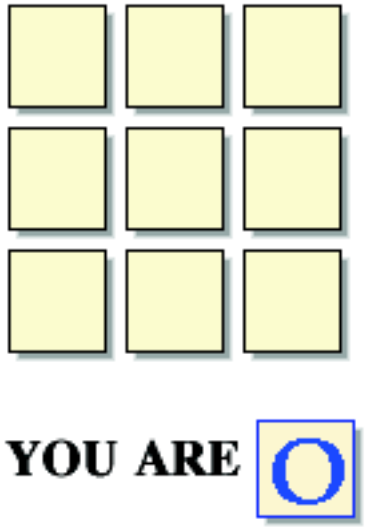
Design Pattern

Characteristics of Get-15

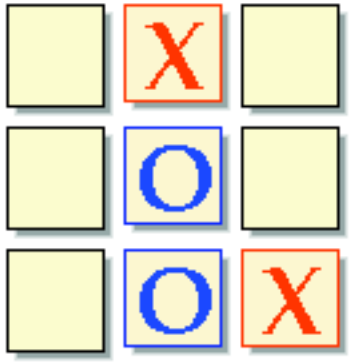
- Hard to play,
- The game is especially hard, if you are not allowed to write anything done.
- Why?
  - All the numbers need to be scanned to see if you have won/lost
  - It is hard to see what the opponent will take if you take a certain number
  - The choice of the number depends on all the previous numbers
  - Not easy to devise an simple strategy

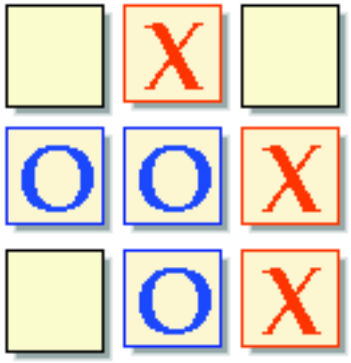
Design Pattern

Another Game: Tic-Tac-Toe

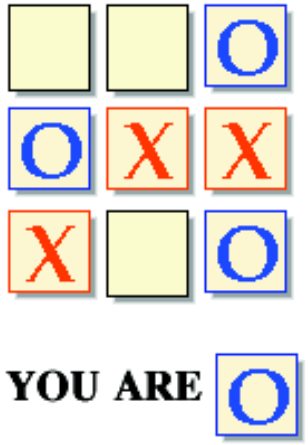


Another Game: Tic-Tac-Toe





Strategy for determining a winning move

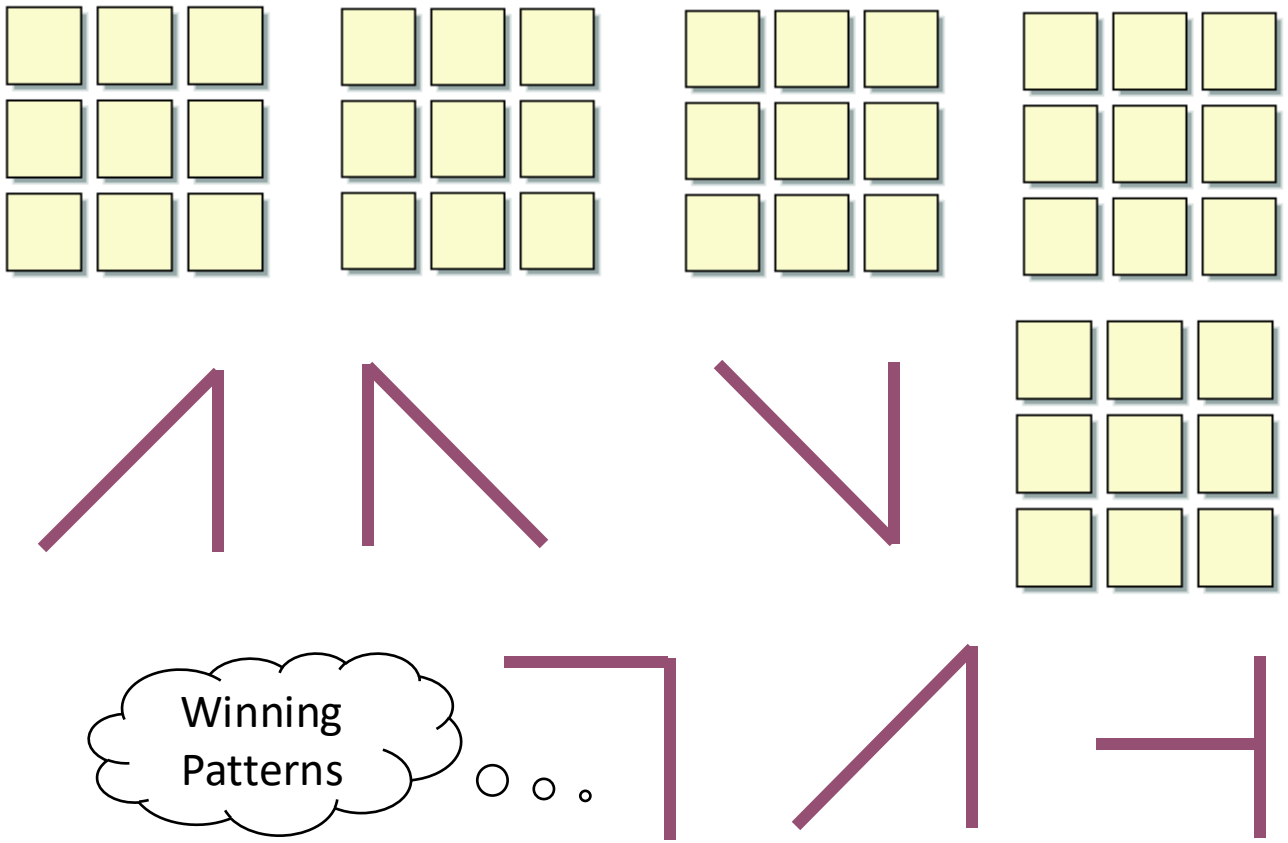


A draw Situation

Design Pattern

Another Game: Tic-Tac-Toe

Winning Situations for Tic-Tac-Toe

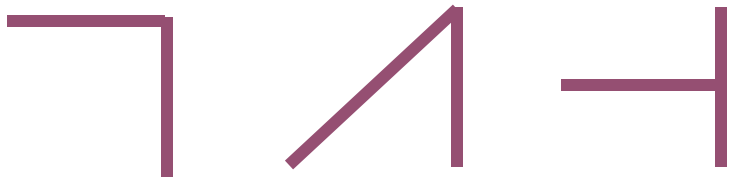


Design Pattern

Another Game: Tic-Tac-Toe

Tic-Tac-Toe is "Easy"

- Why? Reduction of complexity through patterns and symmetries
- **Patterns**: Knowing the following two patterns, the player can anticipate the opponents move.



- Symmetries:
  - The player needs to remember only these three patterns to deal with 8 different game situations

♦ The player needs to memorize only 3 opening moves and their responses

Design Pattern

- Any three numbers that solve the 15 problem also solve tic-tac-toe.
- Any tic-tac-toe solution is also a solution the 15 problem
- To see the relationship between the two games, we simply arrange the 9 digits into the following pattern

Get-15 and Tic-Tac-Toe are identical problems

8	1	6
3	5	7
4	9	2

Object Modeling and design pattern

- During Object Modeling we do many transformations and changes to the object model.
- It is important to make sure the object design model stays simple!
- We use design patterns to keep system models simple.

Modeling Heuristics

Modeling must address our mental limitations:

- Our short-term memory has only limited capacity (7+-2), actual capacity might be lower, around 4±1 chunks (Cowan, 2001)

- Good Models deal with this limitation, because they
  - Do not tax the mind
    - A good model requires only a minimal mental effort to understand
  - Reduce complexity
    - Turn complex tasks into easy ones (by good choice of representation)
    - Use of symmetries
  - Use abstractions
    - Ontologies and taxonomies
  - Have organizational structure:
    - Memory limitations are overcome with an appropriate representation (“natural model”)

Design Pattern

- What are Design Patterns?
  - In software engineering, a **design pattern** is a **reusable, general solution** to a commonly occurring problem in software design.
  - It's not a finished piece of code, but rather a **template** or **blueprint** for solving a problem in a way that balances flexibility, maintainability, and efficiency.
  - Design patterns capture best practices and lessons learned from experienced developers.

- What are Design Patterns?
  - A design pattern describes a problem which occurs over and over again in our environment
  - Then it describes the core of the solution to that problem, in such a way that you can use the this solution a million times over, without ever doing it the same twice

Design Pattern :History

- The idea was picked up by four authors: **Erich Gamma**, **John Vlissides, Ralph Johnson**, and **Richard Helm**. In 1994, they published *Design Patterns: Elements of Reusable Object-Oriented Software*, in which they applied the concept of design patterns to programming.
- The book featured 23 patterns solving various problems of object-oriented design and became a best-seller very quickly.
- Due to its lengthy name, people started to call it "the book by the gang of four" which was soon shortened to simply "the GoF book".

Design Pattern

- **Key Characteristics of Patterns:**
  - **Proven Solutions**: They are tested and refined over time to address recurring challenges
  - **Abstract**: They describe a concept, not specific implementations (they can be adapted to different languages or frameworks)
  - **Shared Vocabulary**: They provide a common language for developers to discuss design decisions (e.g., "Let's use an Observer pattern here").

Design Pattern

In object-oriented development, **design patterns** are template solutions that developers have refined over time to solve a range of recurring problems [Gamma et al., 1994]. A design pattern has four elements:

- A *name* that uniquely identifies the pattern from other patterns.
- A *problem description* that describes the situations in which the pattern can be used. Problems addressed by design patterns are usually the realization of modifiability and extensibility design goals and nonfunctional requirements.
- A *solution* stated as a set of collaborating classes and interfaces.
- A set of *consequences* that describes the trade-offs and alternatives to be considered with respect to the design goals being addressed.

Design Pattern

A design pattern: Example

Name	Adapter Design Pattern
Problem description	Convert the interface of a legacy class into a different interface expected by the client, so that the client and the legacy class can work together without changes.
Solution	An `Adapter` class implements the `ClientInterface` expected by the client. The `Adapter` delegates requests from the client to the `LegacyClass` and performs any necessary conversion.
Consequences	• `Client` and `LegacyClass` work together without modification of neither `Client` nor `LegacyClass`. • `Adapter` works with `LegacyClass` and all of its subclasses. • A new `Adapter` needs to be written for each specialization (e.g., subclass) of `ClientInterface`.

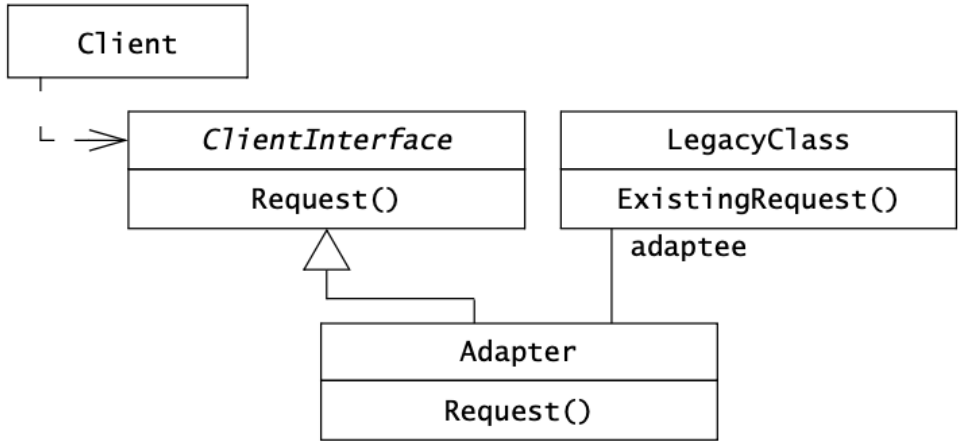


Figure 8-5 An example of design pattern, Adapter (adapted from [Gamma et al., 1994]).

Design Pattern

Why Patterns Matter:

- **Avoid Reinventing the Wheel**: Solve problems using established best practices.
- **Improve Communication**: Say "use a Factory pattern" instead of explaining the entire concept.
- **Enhance Maintainability**: Patterns often lead to cleaner, modular code.
- **Mitigate Anti-Patterns**: Recognize and avoid common pitfalls (e.g., "God Object" or "Spaghetti Code").



Design Pattern

Types of Patterns:

Design patterns are often categorized into three groups:

1.Creational Patterns

1. Focus on object creation mechanisms.
2. Examples: **Singleton**, **Factory**, **Builder**.

2.Structural Patterns

1. Focus on composing classes/objects into larger structures.
2. Examples: **Adapter**, **Decorator**, **Composite**.

3.Behavioral Patterns

1. Focus on communication between objects and responsibility assignment.
2. Examples: **Observer**, **Strategy**, **Command**.

Creational Design Pattern

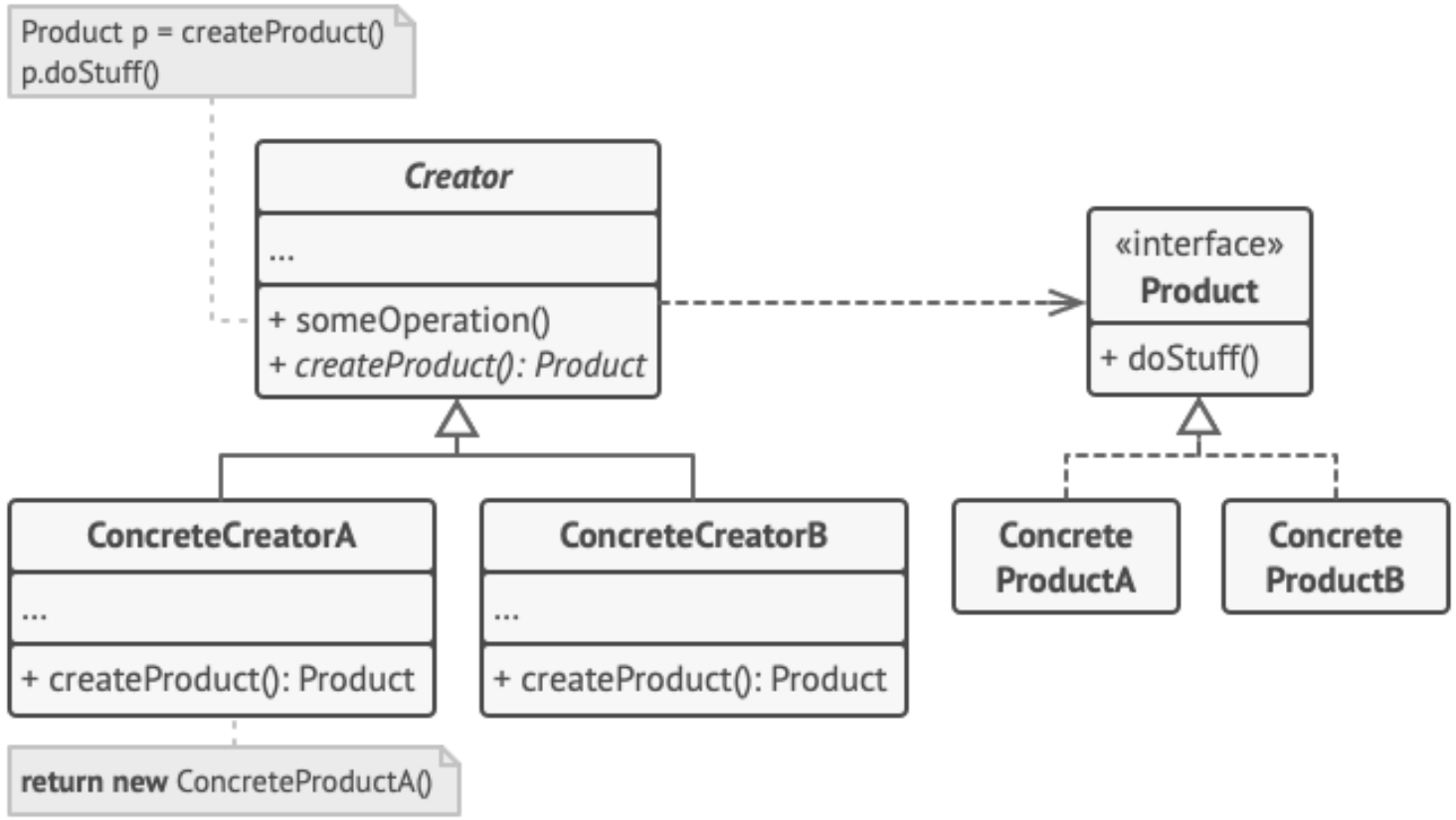
- **Creational patterns** are a category of **design patterns** in software engineering that focus on **object creation mechanisms**.
- They provide flexible and reusable solutions to control how objects are instantiated, initialized, and configured, ensuring systems remain decoupled, maintainable, and scalable. Instead of hardcoding object creation (e.g., using new), these patterns abstract the process to adapt to varying requirements.

Creational Design Pattern

Pattern	Purpose	Example Use Case
Singleton	Ensure a class has **only one instance** with global access.	Database connection pools, logging systems.
Factory Method	Delegate object creation to subclasses via a factory interface.	Creating UI components for different OS.
Abstract Factory	Create **families of related objects** without specifying concrete classes.	Building cross-platform GUI toolkits.
Builder	Construct complex objects **step-by-step**, separating construction from representation.	Configuring a meal (burger, drink, fries).
Prototype	Clone existing objects instead of creating new ones from scratch.	Copying game entities (e.g., NPC templates).

Creational Design Pattern

- The **Factory Pattern** is a **creational design pattern** that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.
- It decouples object creation from the client code, promoting flexibility and adherence to the **Open/Closed Principle**.



Client code uses the Factory Class instead of directly instantiating objects.

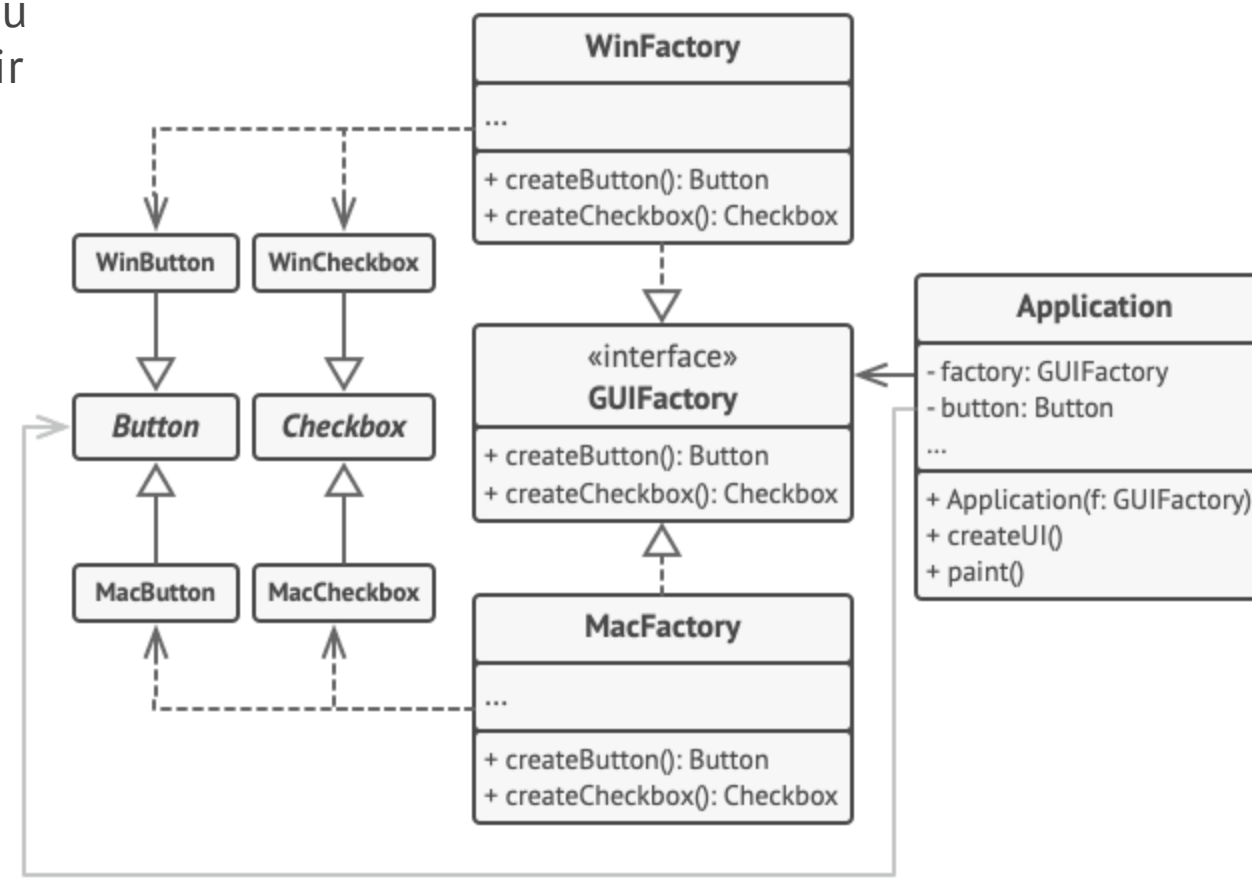
Creational Design Pattern

Abstract Factory is a creational design pattern that lets you produce families of related objects without specifying their concrete classes.

Suppose we are designing a **cross-platform UI framework** that supports **Windows and MacOS**. Each OS has its own **Button** and **Checkbox** styles.

We can use the **Abstract Factory Pattern** to create **OS-specific UI elements** without modifying the core code.

It works like this: when an application launches, it checks the type of the current operating system. The app uses this information to create a factory object from a class that matches the operating system. The rest of the code uses this factory to create UI elements. This prevents the wrong elements from being created.



Structural Design Pattern

- Structural design patterns explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient.
- They simplify relationships between entities by organizing code in a way that makes it easier to extend, reuse, and maintain. These patterns ensure that changes to one part of a system don't ripple through unrelated components.

Intent

- Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly

Composite: Applicability

- Represents part-whole hierarchies of objects.
- Clients ignore the difference between compositions of objects and individual objects.
- Clients treat all objects in the composite structure uniformly.

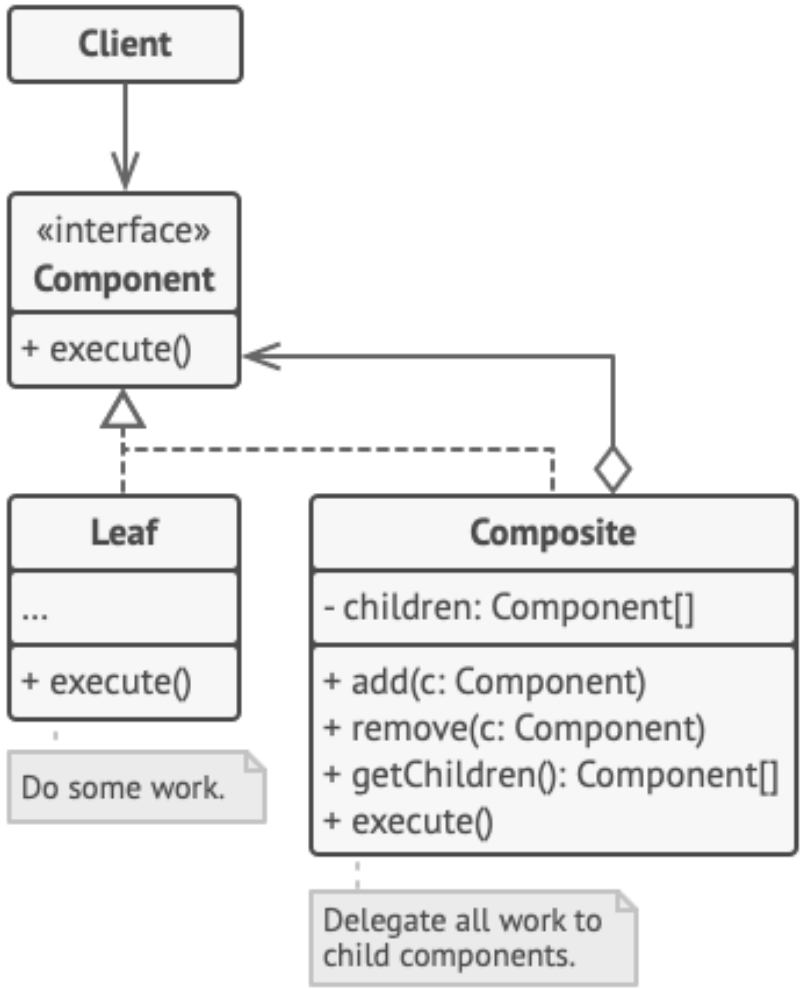
Structural Design Pattern

Common Structural Patterns

Pattern	Purpose	Real-World Analogy
Adapter	Allows incompatible interfaces to work together.	A power plug adapter for different outlets.
Bridge	Separates abstraction from implementation, enabling independent changes.	Remote controls (abstraction) for TVs (implementation).
Composite	Treats individual objects and groups uniformly (tree-like structures).	File system (files + folders as nodes).
Decorator	Dynamically adds responsibilities to objects (wrapper pattern).	Adding toppings to a pizza.
Facade	Provides a simplified interface to a complex subsystem.	A restaurant menu hiding kitchen complexity.
Flyweight	Shares fine-grained objects efficiently (e.g., caching).	Reusing character glyphs in a text editor.
Proxy	Controls access to an object (e.g., lazy loading, security).	A credit card acting as a proxy for cash.

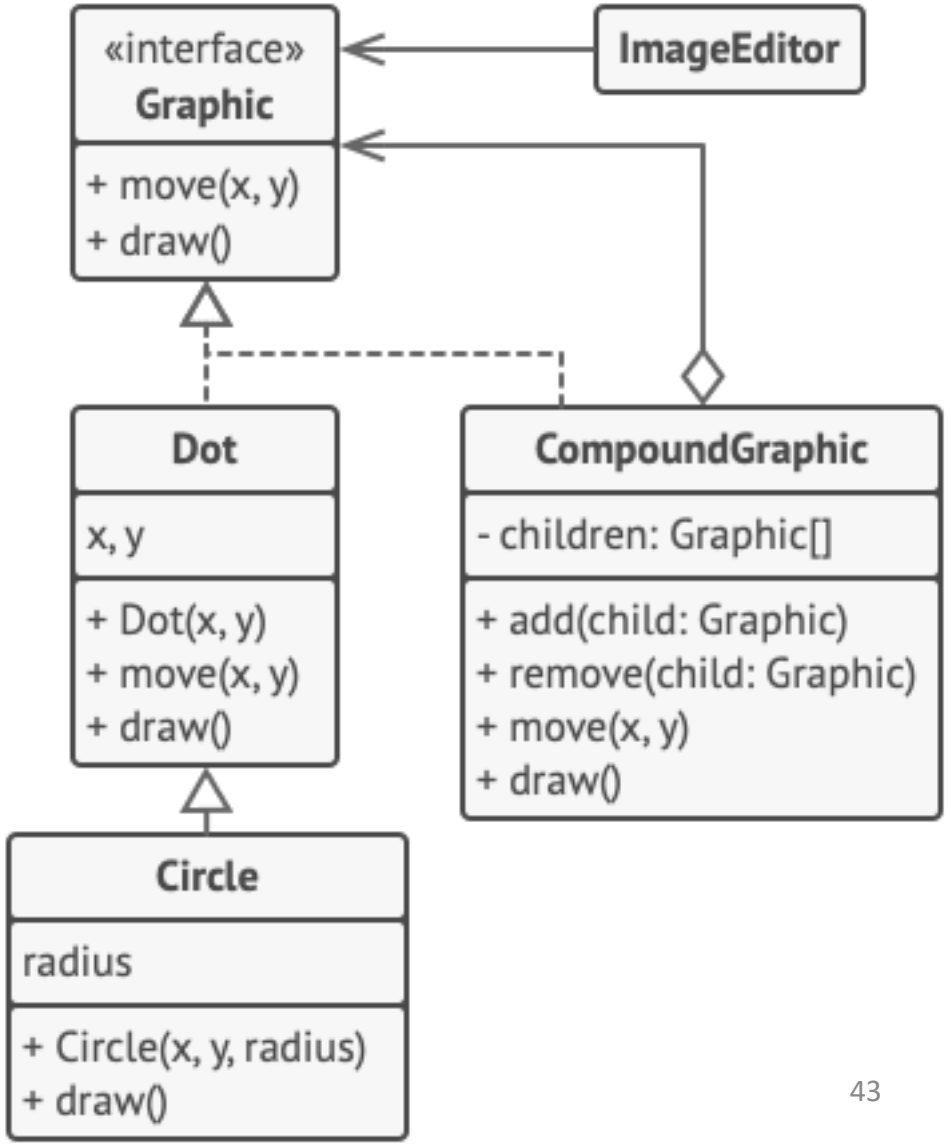
Structural Design Pattern: the Composite Pattern

- Models tree structures that represent part-whole hierarchies with arbitrary depth and width.
- The Composite Pattern lets client treat individual objects and compositions of these objects uniformly



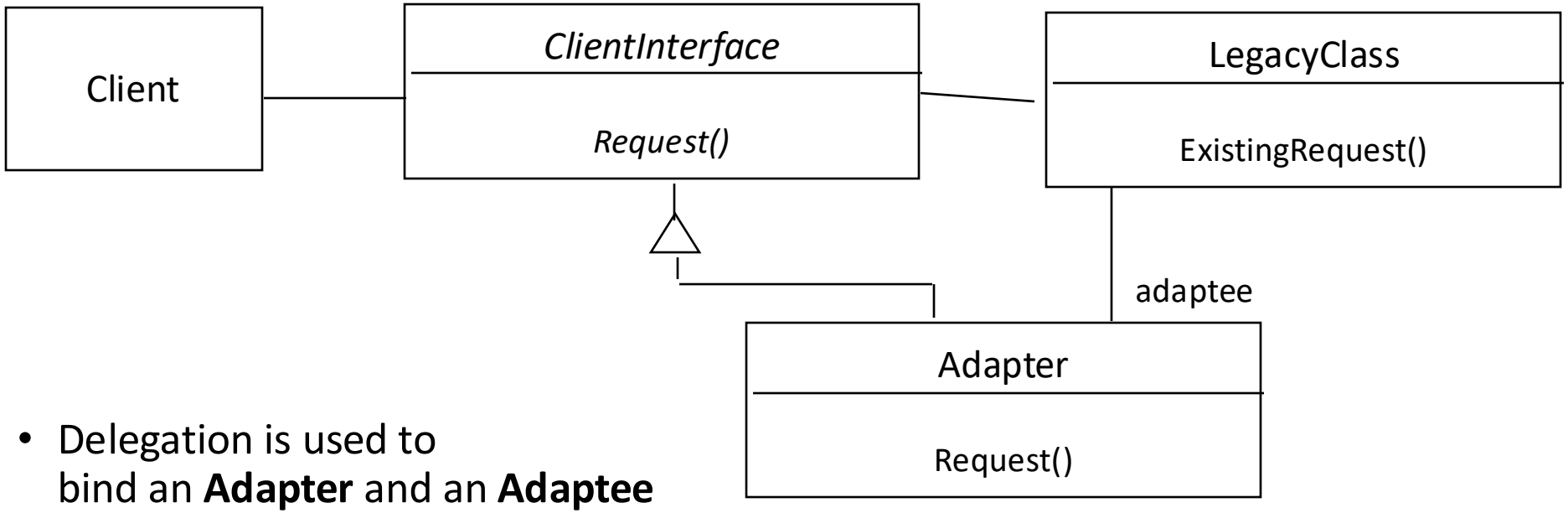
Structural Design Pattern: the Composite Pattern

- The *CompoundGraphic* class is a container that can comprise any number of sub-shapes, including other compound shapes. A compound shape has the same methods as a simple shape. However, instead of doing something on its own, a compound shape passes the request recursively to all its children and "sums up" the result.
- The client code works with all shapes through the single interface common to all shape classes. Thus, the client doesn't know whether it's working with a simple shape or a compound one. The client can work with very complex object structures without being coupled to concrete classes that form that structure.



Structural Design Pattern: the Adapter Pattern

Adapter pattern



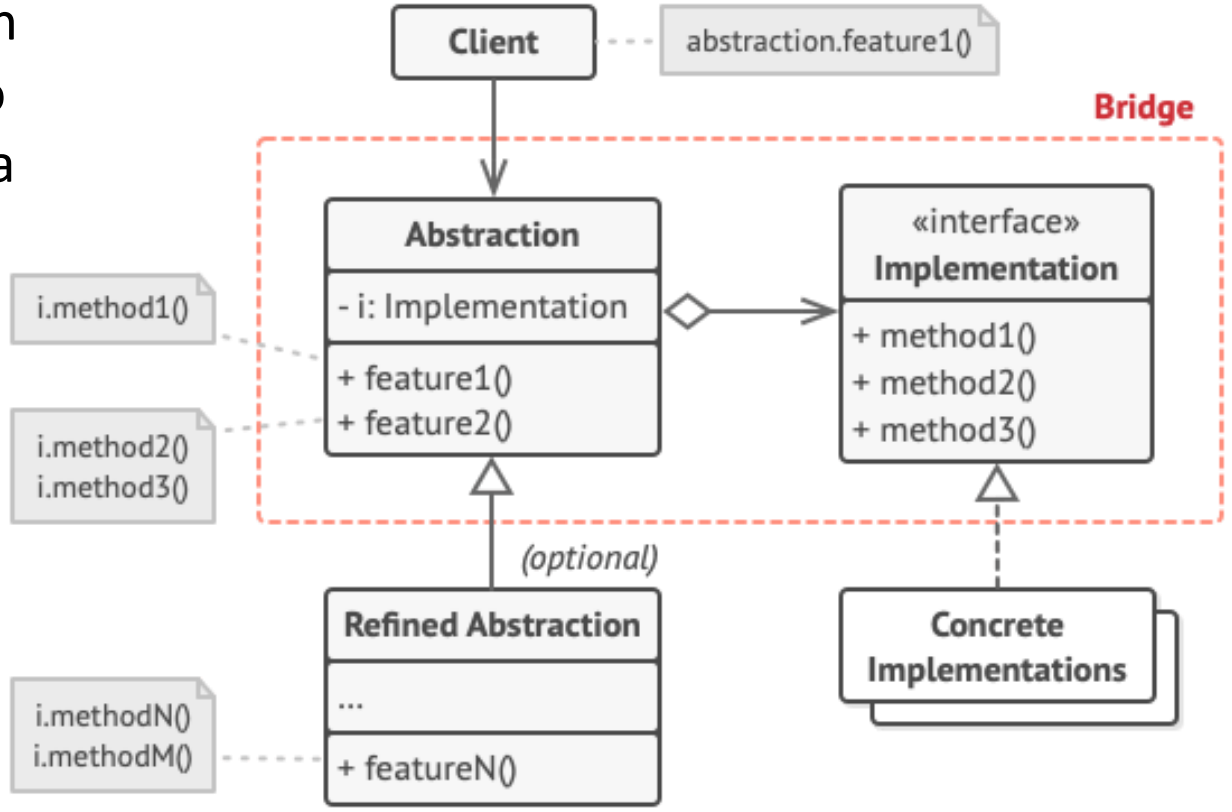
- Delegation is used to bind an **Adapter** and an **Adaptee**
- Interface inheritance is used to specify the interface of the **Adapter** class.
- ***Target*** and **Adaptee** (usually called legacy system) pre-exist the **Adapter.**
- **Target** may be realized as an interface in Java.

Structural Design Pattern: the Adapter Pattern

- The **Adapter Pattern** is a **structural design pattern** that allows two incompatible interfaces to work together by providing a wrapper (adapter) that translates one interface into another.
- An adapter wraps one of the objects to hide the complexity of conversion happening behind the scenes. The wrapped object isn't even aware of the adapter.
- For example, you can wrap an object that operates in meters and kilometers with an adapter that converts all of the data to imperial units such as feet and miles.
- Two adapter patterns:
  - Class adapter:
    - Uses multiple inheritance to adapt one interface to another
  - Object adapter:
    - Uses single inheritance and delegation
- Object adapters are much more frequent. We will only cover object adapters (and call them therefore simply adapters)

Structural Design Pattern: the Bridge Pattern

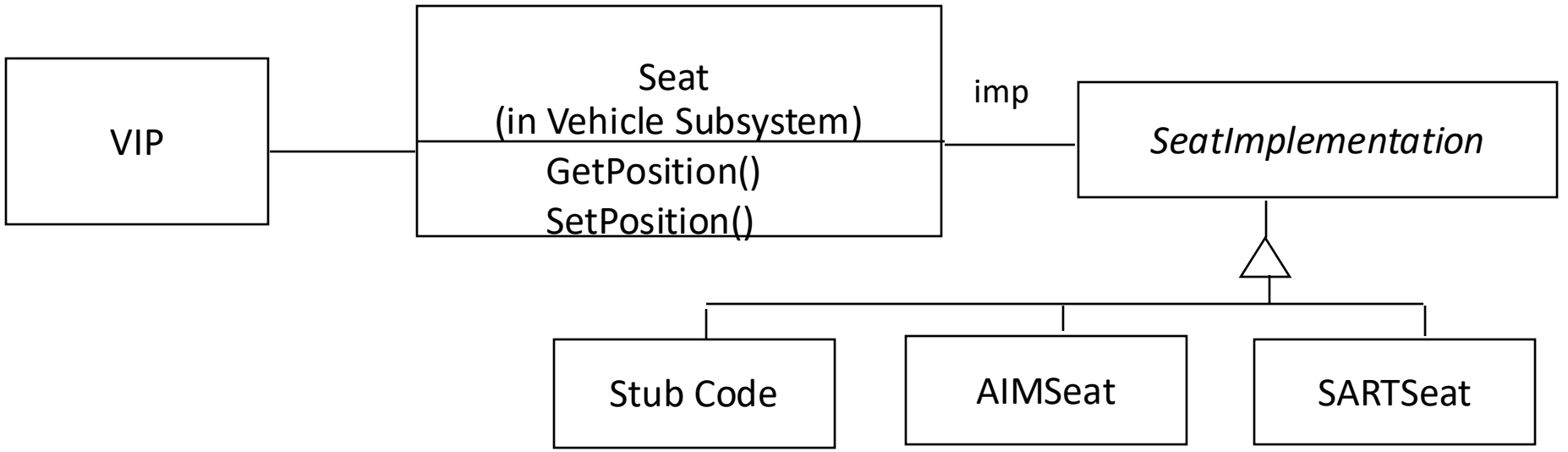
- Use a bridge to “decouple an abstraction from its implementation so that the two can vary independently”. (From [Gamma et al 1995])
- Also know as a Handle/Body pattern.
- Allows different implementations of an interface to be decided upon dynamically.



Structural Design Pattern: the Bridge Pattern

Using a Bridge

- The bridge pattern is used to provide multiple implementations under the same interface.
- Examples: Interface to a component that is incomplete, not yet known or unavailable during testing
- JAMES Project: if seat data is required to be read, but the seat is not yet implemented, known, or only available by a simulation, provide a bridge:



Behavioral Design Pattern

- Behavioral design patterns are concerned with algorithms and the assignment of responsibilities between objects.

Behavioral Design Pattern

- Behavioral design patterns are concerned with algorithms and the assignment of responsibilities between objects.

Behavioral Patterns

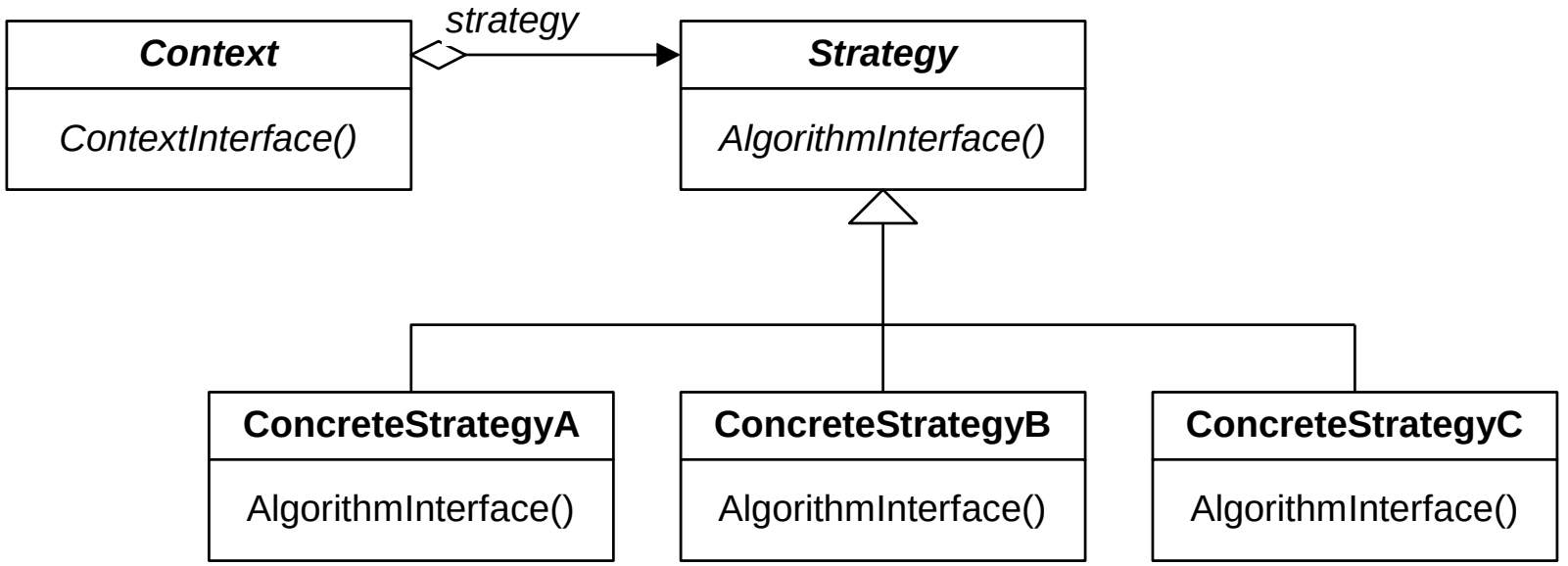
- Observer
- Strategy
- Command
- State
- Visitor

Behavioral Design Pattern : Strategy Pattern

- **Intent:** defines a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

- **Motivation:** when there are many algorithms for solving a problem, hard-wiring all algorithms in client's code may have several problems:
  - Clients get fat and harder to maintain
  - Different algorithms may be appropriate at different time
  - It is difficult to add new algorithms

Behavioral Design Pattern : Strategy Pattern



Behavioral Design Pattern : Strategy Pattern

Participants

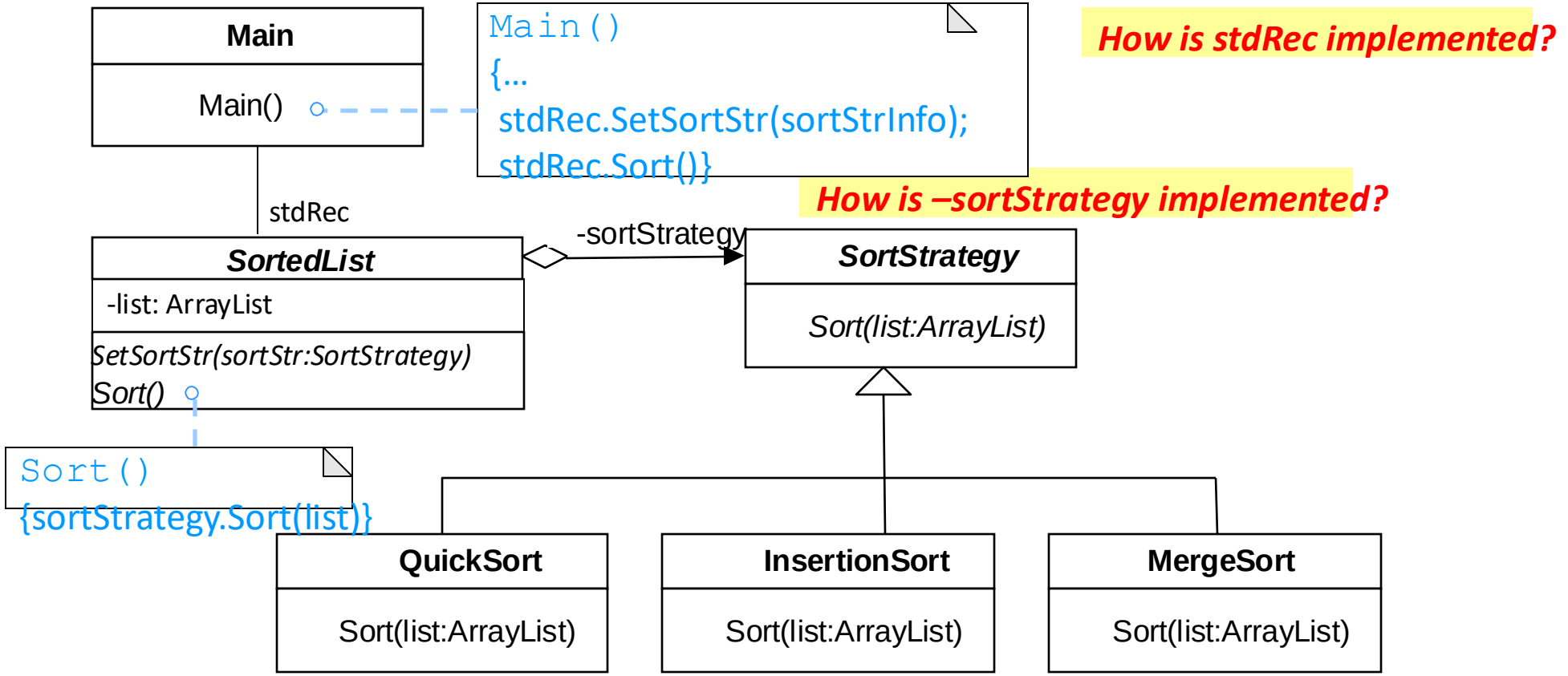
- **Strategy:** declares an interface common to all supported algorithm. Context uses this interface to call the algorithm defined by a ConcreteStrategy.
- **ConcreteStrategy:** implements the algorithm using the Strategy interface
- **Context:** maintains a reference to a Strategy object and defines an interface that let Strategy access its data

Behavioral Design Pattern : Strategy Pattern

- **Requirement:** we want to sort a list of integers using different sorting algorithms, e.g. quick sort, selection sort, insertion sort, etc.
- E.g., {3, 5, 6, 2, 44, 67, 1, 344, ... }
- {1, 2, 3, 5, 6, 44, 67, 344, ... }

- One way to solve this problem is to write a function for each sorting algorithm, e.g.
  - quicksort(int[] in, int[] res)
  - insertionsort(int[] in, int[] res)
  - mergesort(int[] in, int[] res)
- A better way is to use the Strategy pattern

Behavioral Design Pattern : Strategy Pattern



Behavioral Design Pattern :Command Pattern

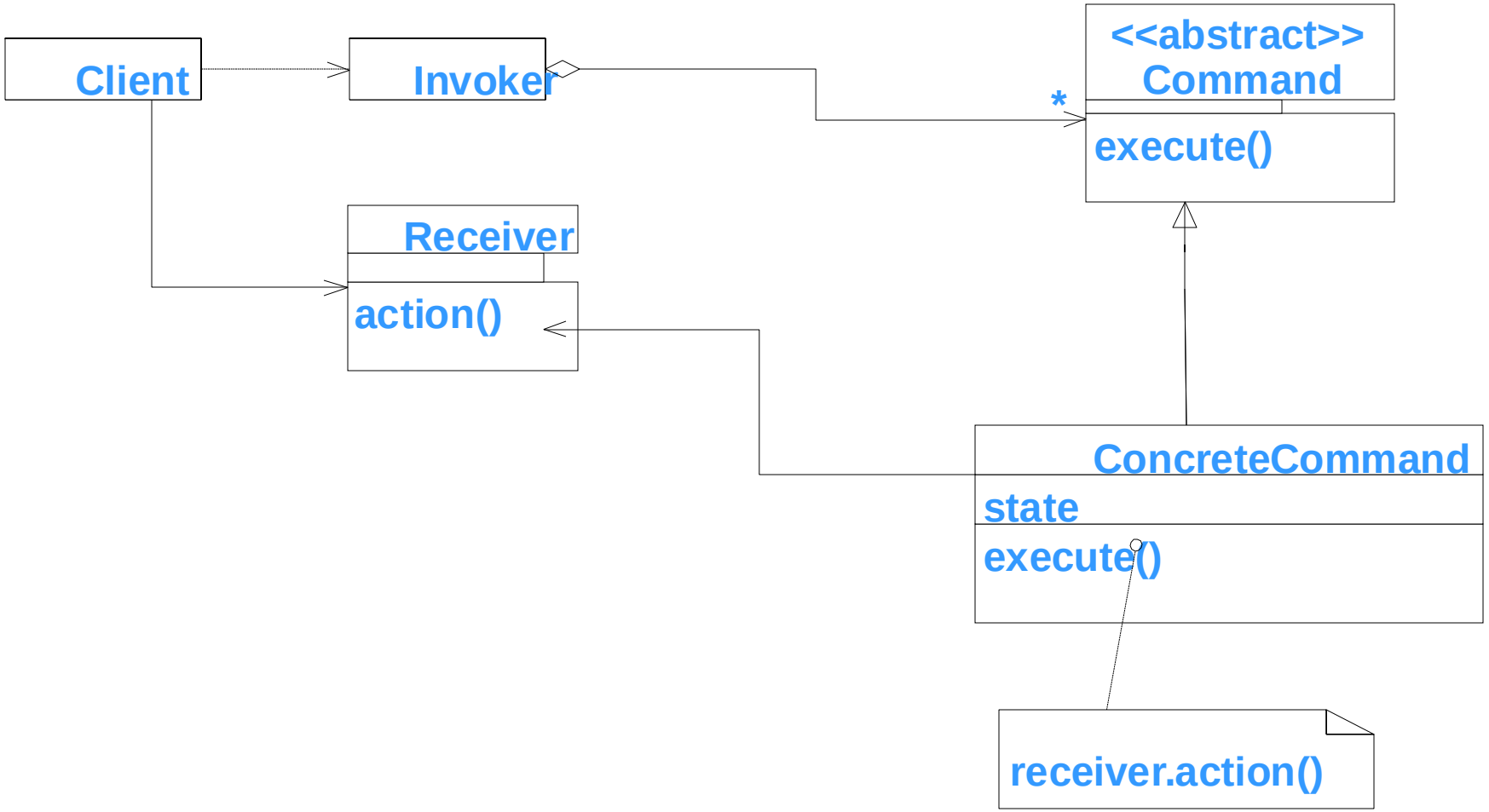
Intent

Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

Applicability

- Parameterize objects by an action
- In place of “callbacks”
- Specify, queue, and execute requests at different times
- Supports undo when Command maintains state information necessary for reversing command.
- Added support for logging Command behavior.
- Support high-level operations built on primitive operations (transactions).

Behavioral Design Pattern :Command Pattern



Behavioral Design Pattern :Command Pattern

Participants

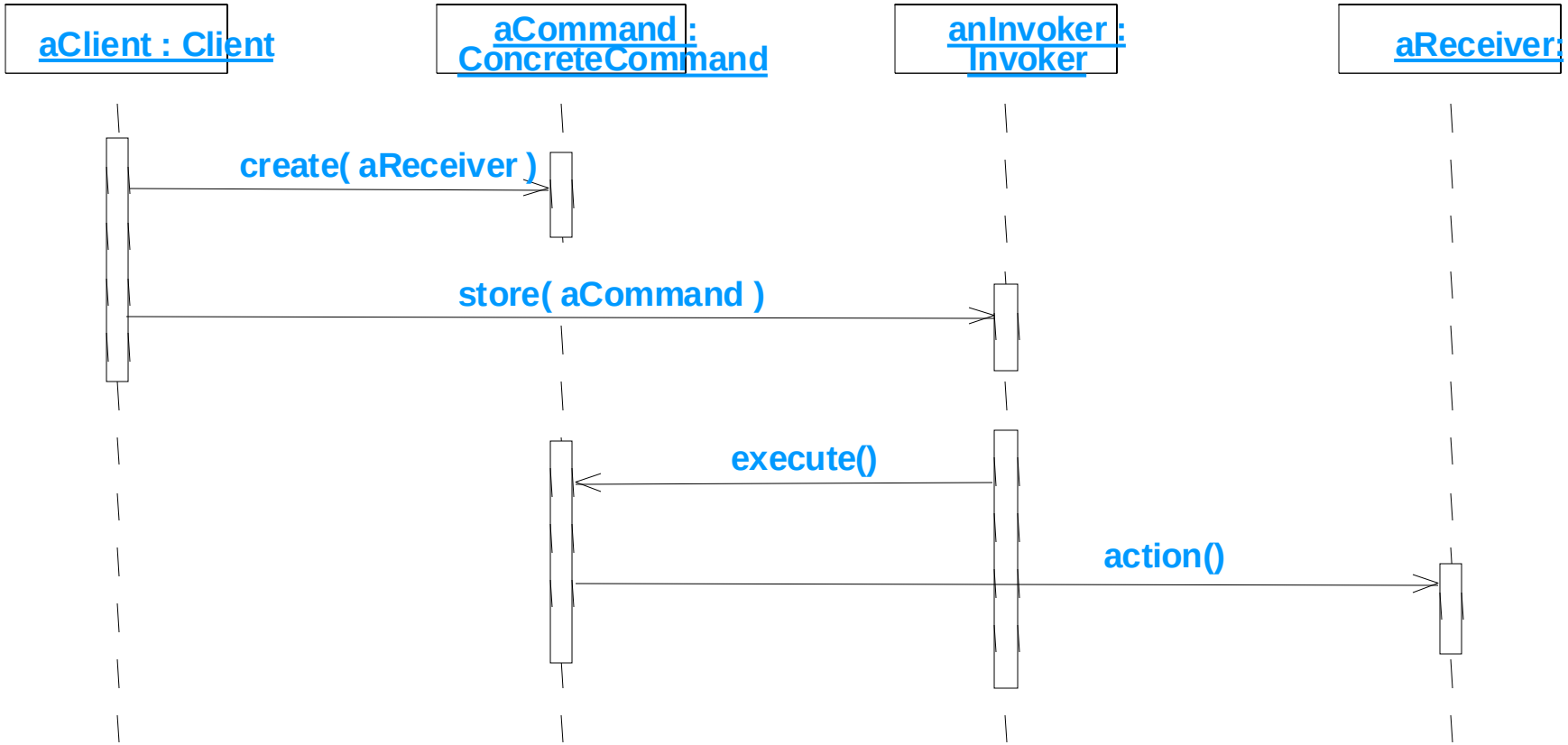
- Command: Declares an interface for executing an operation.
- ConcreteCommand
  - Defines a binding between a Receiver object and an action.
  - Implements execute() by invoking a corresponding operation on Receiver.
- Client (Application): Creates a Command object and sets its Receiver.
- Invoker: Asks the Command to carry out a request.
- Receiver: Knows how to perform the operation associated with a request. Can be any class.

Collaborations

- Creates a ConcreteCommand object and sets its Receiver.
- An Invoker stores the ConcreteCommand.
- Invoker calls *execute()* on command.
- ConcreteCommand invokes operation on its receiver.

Behavioral Design Pattern :Command Pattern

Sequence Diagram



Behavioral Design Pattern :Observer Pattern

Intent

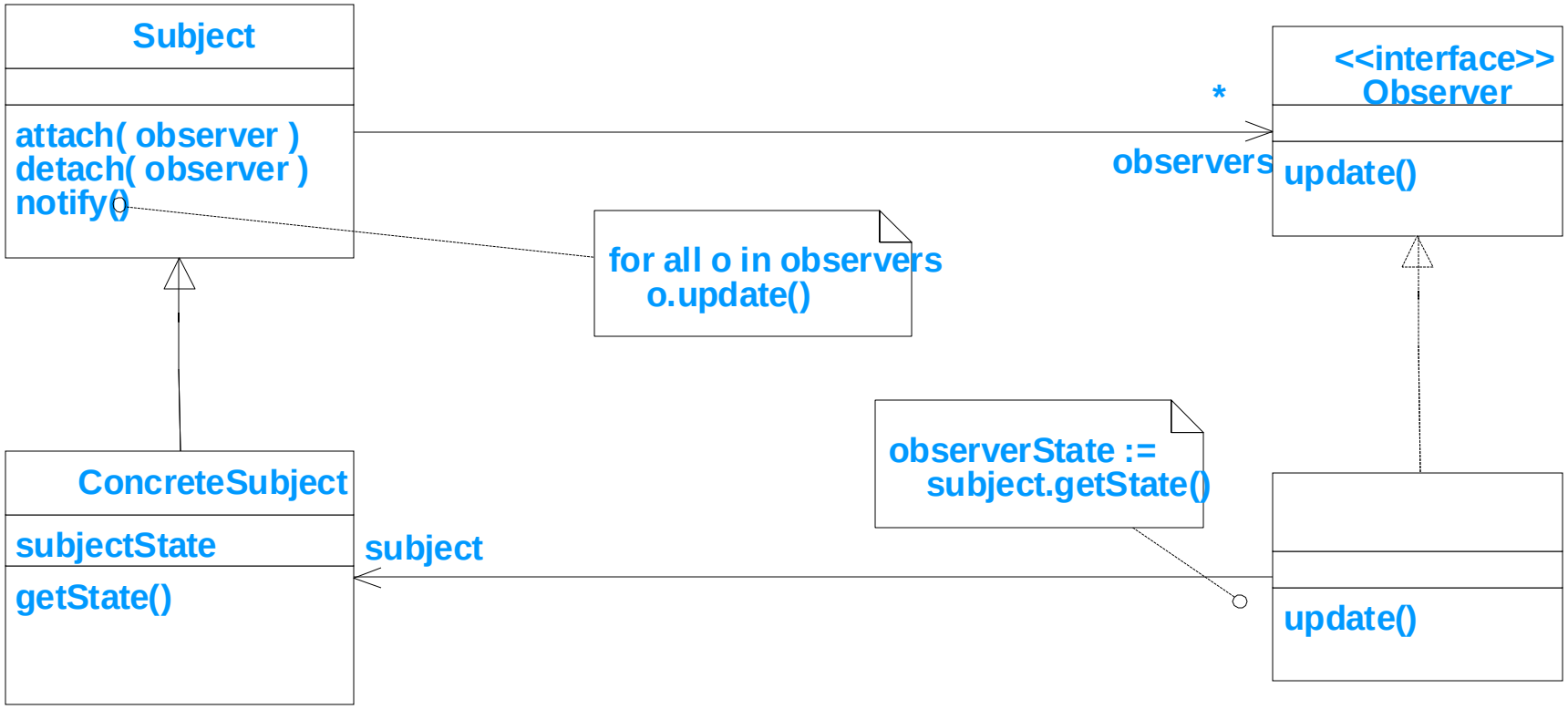
- Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Applicability

- An abstraction has two aspects, one dependent on the other.
- When changing one object requires changing others, and you don't know how many objects need changed.
- When an object needs to notify others without knowledge about who they are.

Behavioral Design Pattern :Observer Pattern

Class Diagram



Behavioral Design Pattern :Observer Pattern

Participants

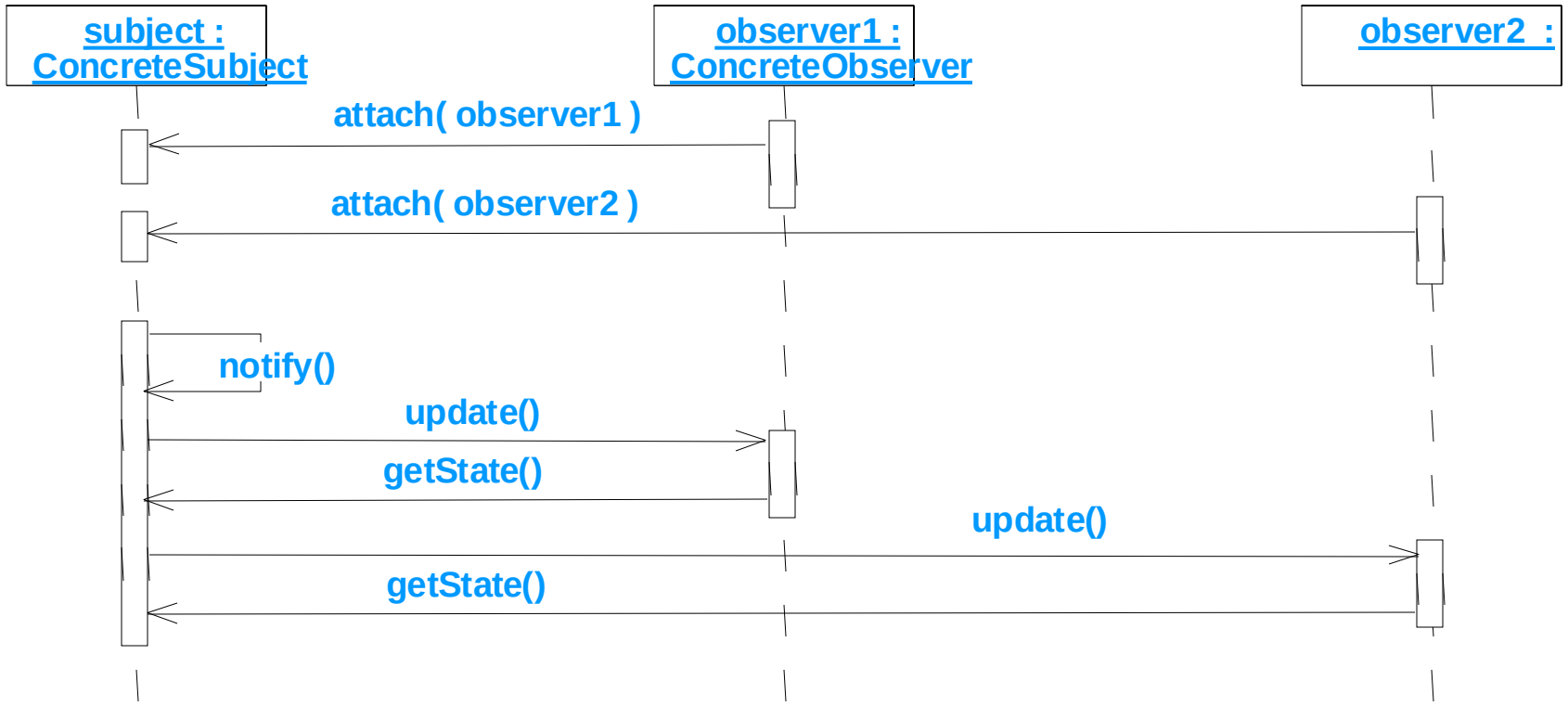
- Subject
  - Knows its observers, but not their “real” identity.
  - Provides an interface for attaching/detaching observers.
- Observer
  - Defines an updating interface for objects that should be identified of changes.
- ConcreteSubject
  - Stores state of interest to ConcreteObserver objects.
  - Sends update notice to observers upon state change.
- ConcreteObserver
  - Maintains reference to ConcreteSubject (sometimes).
  - Maintains state that must be consistent with ConcreteSubject.
  - Implements the Observer interface.

Collaborations

- ConcreteSubject notifies observers when changes occur.
- ConcreteObserver may query subject regarding state change.

Behavioral Design Pattern :Observer Pattern

Sequence Diagram



Selecting and Applying Design Patterns

Heuristics for Pattern Selection
Integrating Patterns into Designs

Selecting and Applying Design Patterns

How to Select *(> 20 and still growing … fast?, more on Internet*)

- Scan Intent Sections
- Study How Patterns Interrelate
- Study Patterns of Like Purpose
- Examine a Cause of Redesign
- Consider What Should Be Variable in Your Design

How to Use

- Read the pattern once through for an overview: *appears trivial, but not*
- Go back and study the structure, participants, and collaborations sections
- Look at Sample Code: *concrete example of pattern in code*
- Choose names for pattern participants
- Define the classes
- Define application specific names for operations in the pattern
- Implement the operations to carry out the responsibilities and collaborations in the pattern

Selecting and Applying Design Patterns

Natural language heuristics for selecting design patterns

Phrase	Design Pattern
• "Manufacturer independence" • "Platform independence"	Abstract Factory
• "Must comply with existing interface" • "Must reuse existing legacy component"	Adapter
• "Must support future protocols"	Bridge
• "All commands should be undoable" • "All transactions should be logged"	Command
• "Must support aggregate structures" • "Must allow for hierarchies of variable depth and width"	Composite
• "Policy and mechanisms should be decoupled". • "Must allow different algorithms to be interchanged at runtime."	Strategy

Managing Reuse

Documenting Reuse Strategies
Assigning Responsibilities for Reusable Components

Managing Reuse

whether design patterns, frameworks, or components, has many technical and managerial advantages:

- ***Lower development effort.***
  - When reusing a solution or a component, many standard errors are avoided. Moreover, in the case of design patterns, the resulting system is more easily extended and more resilient to typical changes. This results in less development effort and reduces the need for human resources.
- ***Lower risk.***
  - When reusing, the typical problems that will be encountered are known and can be anticipated. Moreover, the time needed to adapt the design pattern or to glue the component is also known, resulting in a more predictable development process and fewer risks.
- ***Widespread use of standard terms.***
  - The reuse of a standard set of design patterns and components fosters the use of a standard vocabulary.
- ***Increased reliability.***
  - Reduced development time can lead to an increased testing effort, repetitive use of components can lead to a knowledge base of typical problems to be anticipated, and use of standard terms reduces communication failures.

Managing Reuse

Unfortunately, reuse does not occur spontaneously within a development organization. The main challenges include

- ***NIH (Not Invented Here) syndrome***
  - Since software engineering education (at least until recently) emphasizes mostly the design of new solutions, developers often distrust the reuse of existing solutions, especially when the customization of the solution under consideration is limited or constrained.
- ***Process support***
  - The processes associated with identifying, reusing, and customizing an existing solution are different than those involved in creating a brand new solution. Most software engineering tools and methods are better adapted to creative activities than to reuse.

- ***Training.***
  - Given the lack of knowledge support tools for reuse, training is the single most effective method in establishing a reuse culture. Consequently, the burden of educating developers to specific reusable solutions and components falls on the development organization.

Managing Reuse

Documenting Reuse

- Reuse activities involve two types of documentation:
  - The documentation of the template solution being reused and
  - The documentation of the system that is reusing the solution.

- **Developers can incrementally improve this documentation each time they reuse a solution by adding the following:**
  - ***Reference to a system using the solution.***
    - Minimally, the documentation of the reusable solution should include references to each use.
  - ***Example of use.***
    - Examples are essential for developers to understand the strengths and limitation of the reused solution. Each occurrence of reuse constitutes an example.
  - ***Alternative solutions considered.***
    - In the documentation of the example, developers should indicate which other candidate solutions they discarded and why.
  - ***Encountered trade-offs.***
    - For example, one component may offer an interface that is extensible, and another may deliver better response time.

Managing Reuse

Assigning Responsibility

To foster a reuse culture, an organization needs to make the incentives of reuse as high as possible for the individual developer.

Below are the main roles involved in reuse:

- **Component expert.**
  - The component expert is familiar with using a specific component. The component expert is a developer and usually has received third-party training in the use of the component.
- **Pattern expert.**
  - The pattern expert is the analog of the component expert for a family of design patterns. However, pattern experts are usually self-made and acquire their knowledge from experience.
- **Technical writer.**
  - The technical writer must be aware of reuse and document dependencies between components, design patterns, and the system, as discussed in the previous section.
- **Configuration manager.**
  - In addition to tracking configurations and versions of individual subsystems, the configuration manager must also be aware of the versions of the components that are used.

Managing Reuse

- The technical means of achieving reuse (e.g., inheritance, delegation, design patterns, application frameworks) have been available to software engineers for nearly two decades.
- The success factors associated with reuse are actually not technical, but managerial.
- Only an organization that provides the tools for selecting and improving reusable solutions and the culture to encourage their use can reap the benefits of design and code reuse.